

Chapter 36

The Machine Learning Workflow & Feature Engineering

Turn a business request into a machine learning problem with a clear target, prediction moment, and success measure · split data into training, validation, and test sets, and choose between a random and a time-based split · use cross-validation and read the spread of its scores

Approved chapter · v1

Contents

Chapter 36. The Machine Learning Workflow & Feature Engineering

Why this matters

In plain English

36.1 Framing the problem

36.2 One row per real enquiry

36.3 Train, validation, and test sets

36.4 Cross-validation

36.5 Data leakage

36.6 Feature engineering

36.7 Missing values in machine learning

36.8 scikit-learn pipelines

36.9 Baselines, and the final test

Common mistakes and how to spot them

In the real world: Vikram's 98% model

Tools

The project: a leakage-free feature pipeline for Riverstone lead scoring

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 36. The Machine Learning Workflow & Feature Engineering

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: turn a business request into a machine learning problem with a clear target, prediction moment, and success measure · split data into training, validation, and test sets, and choose between a random and a time-based split · use cross-validation and read the spread of its scores · find data leakage with one question, and measure what each leak does to a score · engineer features from categories, numbers, dates, text, and activity logs · handle missing values so the model learns from them instead of choking on them · build a scikit-learn pipeline that makes leakage from preprocessing impossible · set simple baselines and judge a model against them.

Before you start: Chapter 13 (deduplicating leads and funnels in SQL), Chapter 17 (Python basics), Chapter 18 (pandas), and Chapter 35 (loss, log loss, scaling, and the base-rate benchmark). Chapter 21's idea of a sample helps.

Time needed: 8–12 hours over one to two weeks, including the project.

Tools: Python 3 with pandas, NumPy, and scikit-learn (all free). Everything runs on a laptop in seconds.

Practice data: Riverstone's full CRM export, built by `companion/generate_riverstone_crm.py` : `leads.csv` (12,294 rows), `activities.csv`, and `stage_history.csv`, covering every enquiry from January 2023 to December 2025. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Most machine learning failures at work aren't caused by choosing the wrong algorithm. They're caused by what happens before the algorithm:

- A model scores 98% in the notebook and is useless in production, because one column was filled in *after* the outcome was known.
- A model is tested on customers it has already seen, so the test says nothing about new ones.
- Everything is scaled using the whole dataset, and the test score quietly borrows information from the test set.
- Nobody checks whether the model beats "call referral leads first", a rule a sales manager could apply with no model at all.

This chapter is about the **workflow** that prevents those failures, and about **feature engineering**, the craft of turning raw columns into inputs a model can learn from. Algorithms change every few years; this workflow doesn't. It's also what interviewers probe hardest, because it separates people who have trained models from people who have shipped them.

You'll build it end to end on one real task: scoring Riverstone's incoming sales leads so reps know whom to call first.

In plain English

Think of a school exam.

A teacher wants to know whether students have *learned* the subject, not whether they've memorized last year's paper. So the teacher keeps the real exam questions locked away (the **test set**), lets students practice on past papers (the **training set**), and gives a mock exam halfway through to see who's ready (the **validation set**). If the real questions leak before the exam, every score goes up and means nothing: that's **data leakage**.

Students who revise with good notes do better than students who read the textbook raw. Summarizing a chapter into a few key points, turning dates into "which term", grouping similar topics together: that's **feature engineering**.

The teacher also runs the exam the same way every year: same rules, same timing, same marking. Writing that procedure down so nobody can accidentally hand out the answers is what a **pipeline** does.

And before praising a student for scoring 60%, the teacher checks what a student would score by guessing. That's a **baseline**.

36.1 Framing the problem

From a request to a question a model can answer

Anita Rao, Riverstone's Sales Head, has a problem. Enquiries have grown from about 290 a month in 2023 to about 400 in 2025, most of them from a B2B marketplace listing. Her three sales executives and the inside sales desk can't call everyone quickly. Her request: *"Can we tell which leads are worth calling first?"*

That's a business request, not yet a machine learning problem. Before touching data, pin down five things:

Decision	Riverstone lead scoring
Unit (one row = ?)	One real enquiry (duplicates removed)
Target (what we predict)	Won within 90 days of arriving: 1 or 0

Decision	Riverstone lead scoring
Prediction moment (when we predict)	24 hours after the lead arrives, when the morning call list is made
Allowed information	Only what is recorded in the CRM by that moment
Success measure	Better ranking than the current rule, and better probabilities than the base rate (log loss, Chapter 35)

This is a **supervised learning** problem: you have past examples with known answers (won or lost) and want to predict the answer for new ones. Because the target has two values, it's a **binary classification** problem. Predicting a number, such as order value, would be **regression**. Chapter 37 covers the algorithms for both.

The **prediction moment** is the decision people most often skip, and it's the one that prevents most leakage. Every feature must pass one test: *would this value be in the CRM 24 hours after the lead arrived?*

Watch out: a vague target makes a vague model. "Good lead" isn't a target. "Won" isn't quite one either, until you say *by when*. Without the 90-day window, a lead from last week that will be won next month counts as "lost" today, and the model learns that recent leads are bad.

A first look at the data

```
import numpy as np
import pandas as pd

leads = pd.read_csv("leads.csv", parse_dates=["created_at"])
print(leads.shape)
print(
    leads[
        [
            "lead_id",
            "created_at",
            "source",
            "segment",
            "company_size",
            "est_quantity",
            "status",
        ]
    ].head(4)
    .to_string(index=False)
)
print(leads["status"].value_counts().to_string())
```

```
(12294, 17)
lead_id      created_at      source      segment  company_size  est_quantity  status
100001 2023-01-01 08:39:23 Marketplace    Retail        11-50         78.0    Lost
100002 2023-01-01 08:58:54      Website    Retail         1-10         65.0    Lost
100003 2023-01-01 09:16:29 Marketplace      NaN        200+        1009.0    Won
100004 2023-01-01 09:38:54      Referral  Wholesale        11-50         103.0    Lost

status
Lost      10365
Open       1065
Won         864
```

This is Riverstone's full CRM, much bigger than the `leads` table in Chapter 13's one-year database, which holds only the enquiries reps logged themselves. The export also includes every marketplace and web-form lead handled by the inside sales desk, a small team that works through high-volume enquiries. It was taken at the end of 31 December 2025. It has 12,294 rows and 17 columns. Of these, 1,065 leads are still **Open**, so their outcome isn't known yet.

Only use leads whose outcome is settled

Leads that arrived in the last 90 days of 2025 haven't had their full 90 days. Some that will be won are still open, but the ones that were lost quickly are already marked Lost. Look at the win rate among closed leads by the quarter they arrived:

```
closed = leads[leads["status"] != "Open"]
quarter = closed["created_at"].dt.to_period("Q")
by_quarter = closed.groupby(quarter)["status"].apply(lambda s: (s == "Won").mean())
print((by_quarter.tail(6) * 100).round(1).to_string())
```

```
created_at
2024Q3      9.0
2024Q4      7.4
2025Q1      7.2
2025Q2      5.6
2025Q3      8.7
2025Q4     12.5
Freq: Q-DEC
```

The win rate jumps to 12.5% in the last quarter of 2025. That's not a sales miracle. Fewer than a quarter of that quarter's leads have closed by the export, and the ones that closed are the fast outcomes: a win often closes within weeks, while many lost leads stay open until they're closed by default at 90 days. Training on those rows would teach the model that October-to-December leads are unusually good. (With other closing habits, the bias runs the other way.) So the rule is: **only use leads created at least 90 days before the export**, which means on or before 2 October 2025.

This problem, where recent outcomes are incomplete, is called **censoring** (or *right-censoring*), and it appears in churn, credit default, and anything with a waiting period.

36.2 One row per real enquiry

Machine learning needs a clean table: one row per unit, one column per feature, and a target column. Chapter 13 removed duplicate web-form submissions in SQL; here's the same rule in pandas, plus the city spellings.

```
leads["email_norm"] = leads["email"].str.lower().str.strip()
leads = leads.sort_values("created_at")
gap = leads.groupby("email_norm")["created_at"].diff()
is_duplicate = gap.notna() & (gap <= pd.Timedelta(days=2))
print("duplicate submissions removed:", is_duplicate.sum())
leads = leads[~is_duplicate].copy()

export_time = pd.Timestamp("2025-12-31 23:59")
leads = leads[leads["created_at"] <= export_time - pd.Timedelta(days=90)].copy()
leads["won"] = (leads["status"] == "Won").astype(int)

city_fix = {
    "bombay": "Mumbai",
    "mumbai.": "Mumbai",
    "bangalore": "Bengaluru",
    "b'lore": "Bengaluru",
    "new delhi": "Delhi",
    "delhi ncr": "Delhi",
    "poona": "Pune",
    "madras": "Chennai",
    "calcutta": "Kolkata",
    "panaji": "Goa",
}

city_lower = leads["city"].str.strip().str.lower()
leads["city_clean"] = city_lower.map(city_fix).fillna(city_lower.str.title())

print(f"{len(leads):,} leads, {leads['won'].sum()} won ({leads['won'].mean():.1%})")
print(
    leads["city_clean"].nunique(),
    "cities after cleaning (was",
    leads["city"].nunique(),
    "spellings)",
    sep=" ",
)
```

```
duplicate submissions removed: 271
10,701 leads, 826 won (7.7%)
17 cities after cleaning (was 35 spellings)
```

How it works:

- Emails are lower-cased and trimmed first, so `METR012@...` and `metro12@...` count as the same person.
- `groupby(...).diff()` gives the time since the same email's previous lead. A repeat within two days is a resubmission of the same enquiry, not a new one.
- The city map sends each alternative spelling to one name; `.str.title()` fixes case for the rest ("pune" becomes "Pune").

Reconcile. 12,294 rows – 271 duplicates = 12,023 real enquiries, of which 10,701 arrived by 2 October 2025. They include 826 wins, a win rate of **7.7%**. That's the base rate: out of every 13 leads, about one is won.

Real-world note. The generator planted 268 duplicates; the two-day rule removed 271. Three genuine enquiries from the same email happened to fall within two days of each other. Every deduplication rule has edge cases; write the rule down, and count what it removes.

36.3 Train, validation, and test sets

Why three sets

A model's score on the data it trained on says little about how it will do on new data. A flexible enough model can memorize the training rows. So you hold data back:

- The **training set** is what the model learns from.
- The **validation set** is what you use to compare choices: which features, which settings, which model. You look at it many times.
- The **test set** is locked away and used **once**, at the very end, to report how the final model does on data nobody tuned against.

Every time you check the validation score and change something, a little information about the validation set leaks into your decisions. After fifty rounds of "try something, check validation", the validation score is optimistic. The untouched test set is what keeps you honest.

Split by time when the future is what you'll predict

The usual default is a **random split**: shuffle the rows and cut. That's fine when rows are independent and the world doesn't change. Riverstone's model will score *next month's* leads using a model trained on the past, and the mix of leads is changing. So split by **time**, the way the model will be used:

```

train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]

for name, part in [
    ("train (2023-2024)", train),
    ("valid (Jan-Jun 2025)", valid),
    ("test (Jul-2 Oct 2025)", test),
]:
    print(f"{name:<22} {len(part):>6,} leads    win rate {part['won'].mean():.2%}")

yearly = leads.groupby(leads["created_at"].dt.year)
print(
    "\nmarketplace share:  ",
    (yearly["source"].apply(lambda s: (s == "Marketplace").mean()) * 100)
    .round(1)
    .to_dict(),
)
print(
    "marketplace win rate:",
    (
        leads[leads["source"] == "Marketplace"]
        .groupby(leads["created_at"].dt.year)["won"]
        .mean()
        * 100
    )
    .round(2)
    .to_dict(),
)

```

```

train (2023-2024)      7,291 leads    win rate 7.90%
valid (Jan-Jun 2025)   2,225 leads    win rate 6.56%
test (Jul-2 Oct 2025)  1,185 leads    win rate 8.78%

marketplace share:     {2023: 30.1, 2024: 34.9, 2025: 44.5}
marketplace win rate:  {2023: 2.65, 2024: 3.23, 2025: 1.52}

```

The model learns from the past and is judged on later leads

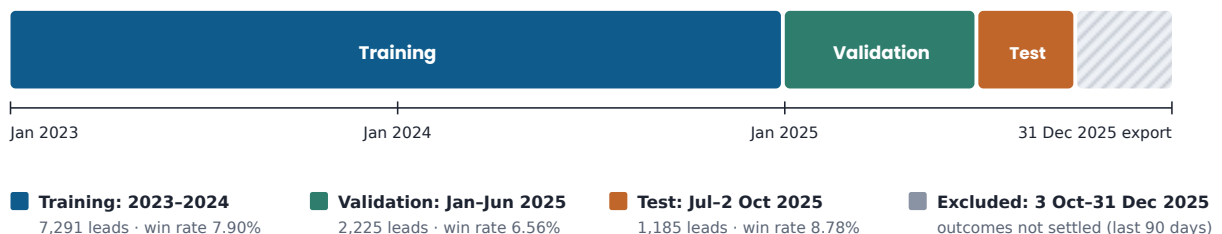


Figure 36.1 – Riverstone's time-based split. The model learns from the past and is judged on later leads, the way it will be used. The last 90 days are left out because their outcomes aren't settled.

Reading it. The world did change. The marketplace's share of leads rose from 30.1% in 2023 to 44.5% in 2025, and its win rate halved in 2025, to 1.52%, after Riverstone switched to a cheaper listing plan. A time split puts that change where it belongs: in the future the model must cope with. A random split would mix 2025 leads into training and hide it.

Watch out: scores from different splits can't be compared. You'll see in section 36.9 that on this data, a random split's score happens to be *lower* than the time split's. The 2025 validation set is full of marketplace leads that are clearly weak and rank low without difficulty, and a score depends on the mix of the rows it's measured on. The time split isn't chosen because it gives a better or worse number; it's chosen because it asks the right question. Compare models only on the **same** split.

The rows for the three sets are chosen now and don't change for the rest of the chapter. The test set isn't used again until section 36.9.

36.4 Cross-validation

A single validation set gives a single number, and with 2,225 leads and about 150 wins, that number is noisy. **Cross-validation** reuses the training data to get several estimates.

In **k-fold cross-validation**, the training rows are split into k equal parts (**folds**). The model is trained k times, each time holding out one fold for scoring and training on the rest. You get k scores; their average is the estimate and their spread shows how much it wobbles.

- **Stratified k-fold** keeps the win rate the same in every fold. With a 7.7% win rate, a plain random fold could get noticeably fewer wins than average; stratifying prevents that. Use it for classification.
- **Time series split** trains on earlier folds and tests on the next one, always forward in time. Use it when order matters.

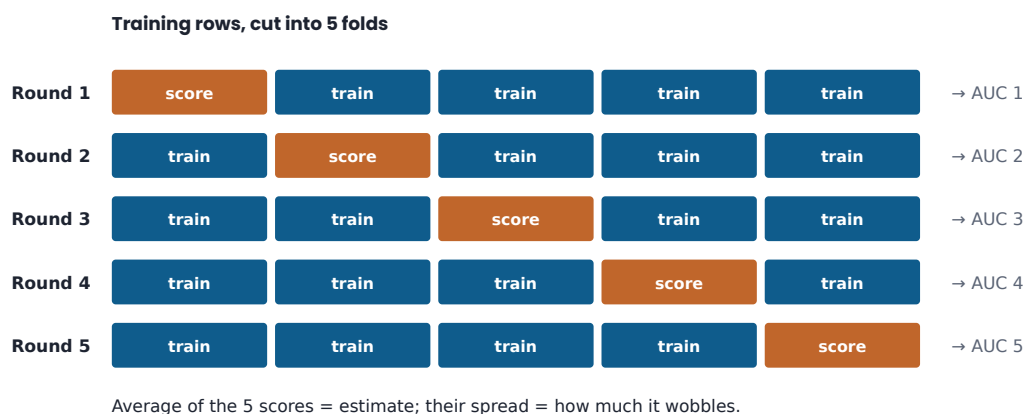


Figure 36.2 — Five-fold cross-validation. Every row is scored exactly once, by a model that didn't train on it.

Cross-validation needs a model to evaluate. Here's a first, deliberately simple model: logistic regression (Chapter 37) on a few columns, wrapped in a pipeline. Section 36.8 explains every piece of the pipeline; for now, treat it as "prepare the columns, then fit the model".

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, TimeSeriesSplit, cross_val_score

def make_model(categorical, numeric):
    prepare = ColumnTransformer(
        [
            (
                "cat",
                Pipeline(
                    [
                        (
                            "fill",
                            SimpleImputer(strategy="constant", fill_value="Missing"),
                        ),
                        ("onehot", OneHotEncoder(handle_unknown="ignore")),
                    ]
                ),
                categorical,
            ),
            (
                "num",
                Pipeline(
                    [
                        ("fill", SimpleImputer(strategy="median", add_indicator=True)),
                        ("scale", StandardScaler()),
                    ]
                ),
                numeric,
            ),
        ]
    )
    return Pipeline(
        [("prepare", prepare), ("model", LogisticRegression(max_iter=1000))]
    )

first_cats = ["source", "segment", "company_size"]
first_nums = ["website_visits"]
X_first, y_train = train[first_cats + first_nums], train["won"]

folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=36)
scores = cross_val_score(
    make_model(first_cats, first_nums), X_first, y_train, cv=folds, scoring="roc_auc"
)
print(
    "stratified 5-fold AUC:",
    scores.round(3),
    " mean",
    scores.mean().round(3),
    " sd",
    scores.std().round(3),
)

forward = cross_val_score(
    make_model(first_cats, first_nums),
    X_first,
    y_train,
    cv=TimeSeriesSplit(n_splits=5),
    scoring="roc_auc",
)

```

```
)
print("time series split AUC:", forward.round(3), " mean", forward.mean().round(3))

stratified 5-fold AUC: [0.734 0.746 0.743 0.733 0.746] mean 0.741 sd 0.006
time series split AUC: [0.68 0.749 0.775 0.742 0.735] mean 0.736
```

How it works:

- `cross_val_score` runs the whole train-and-score loop for you: five fits, five scores. Because the full pipeline is passed in, the fitting and scaling are re-learned inside each fold, from that fold's training rows only. That detail matters in section 36.5.
- `shuffle=True, random_state=36` makes the folds random but repeatable. Every random step in this book has a fixed seed, so you get the same numbers.
- `TimeSeriesSplit` needs rows in time order, which `train` already is (it was sorted by `created_at` in section 36.2). Its first fold trains on only the earliest sixth of the data.

AUC, briefly. The score here is **ROC-AUC** (area under the ROC curve). It has a plain meaning: *pick one won lead and one lost lead at random; AUC is the chance the model gives the won lead the higher score.* 0.5 is a coin toss; 1.0 is perfect ranking. It measures ranking, which is exactly what a call list needs. Chapter 39 builds the ROC curve and explains AUC's limits; log loss (Chapter 35) measures whether the probabilities themselves are right, and this chapter uses both.

Reading it. Even with four simple columns, the model ranks leads far better than chance, around 0.74. The five stratified folds agree closely here (a standard deviation of 0.006); the time-series folds vary much more, from 0.68 for the first fold, which trains on only the earliest sixth of the data, to 0.775. That spread is the honest size of your uncertainty: a change to the model that moves the mean score by 0.005 is noise, not progress.

36.5 Data leakage

The one question

Data leakage means information the model won't have when it's used gets into training or evaluation. Leakage makes scores look better than reality, sometimes spectacularly. Almost every leak is caught by asking, for each feature and each step: **would this be known at the prediction moment?**

Leakage comes in two kinds:

1. **Target leakage:** a feature is recorded after the outcome, or because of it.
2. **Train-test contamination:** information from the validation or test rows gets into training, usually through preprocessing.

The CRM export contains three columns that look like excellent features. Test each against the prediction moment, 24 hours after arrival:

Column	What it records	Known at 24 hours?
<code>quote_sent_date</code>	When a quote was sent	No. Quotes go to qualified leads, days or weeks later
<code>days_in_pipeline</code>	Days from arrival to closing	No. Only known once the lead is closed
<code>first_response_hours</code>	Hours until the first contact	Only if the answer is 24 or less. A value of 60 means "still not contacted at 24 hours", which is known; the exact 60 isn't

Now measure what each does to the score. The legitimate features are built in section 36.6; to keep this section self-contained, the code below adds those features first and then runs one model with each suspicious column added.

```

def add_features(df):
    df = df.copy()
    text = df["enquiry_text"].str.lower()
    df["text_strong"] = text.str.contains(
        "bulk|tender|urgent|monthly|new outlet"
    ).astype(int)
    df["text_weak"] = text.str.contains(
        "price list|sample|checking rates|catalogue|price please"
    ).astype(int)
    df["free_email"] = df["email_norm"].str.endswith("@gmail.com").astype(int)
    cap = 50_000 # quantities above this are typos
    df["log_quantity"] = np.log1p(df["est_quantity"].where(df["est_quantity"] <= cap))
    df["inside_desk"] = (df["owner_id"] == 9).astype(int)
    df["responded_24h"] = (df["first_response_hours"] <= 24).astype(int)
    df["has_quote"] = df["quote_sent_date"].notna().astype(int)
    return df

activities = pd.read_csv("activities.csv", parse_dates=["activity_at", "logged_at"])
deadline = leads.set_index("lead_id")["created_at"] + pd.Timedelta(hours=24)
visible = activities[activities["logged_at"] <= activities["lead_id"].map(deadline)]
leads["activities_24h"] = (
    leads["lead_id"].map(visible.groupby("lead_id").size()).fillna(0)
)

leads = add_features(leads)
train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]

cats = ["source", "segment", "city_clean", "company_size", "product_interest"]
nums = [
    "log_quantity",
    "website_visits",
    "free_email",
    "text_strong",
    "text_weak",
    "inside_desk",
    "activities_24h",
    "responded_24h",
]

from sklearn.metrics import roc_auc_score, log_loss

def score(categorical, numeric, fit_rows=None, eval_rows=None):
    fit_rows = train if fit_rows is None else fit_rows
    eval_rows = valid if eval_rows is None else eval_rows
    model = make_model(categorical, numeric).fit(
        fit_rows[categorical + numeric], fit_rows["won"]
    )
    p = model.predict_proba(eval_rows[categorical + numeric])[:, 1]
    return roc_auc_score(eval_rows["won"], p), log_loss(eval_rows["won"], p)

print("honest features:          AUC %.3f  log loss %.4f" % score(cats, nums))
for leak in ["has_quote", "days_in_pipeline", "first_response_hours"]:
    print(f"+ {leak}<27} AUC %.3f  log loss %.4f" % score(cats, nums + [leak]))

```

honest features:	AUC 0.823	log loss 0.1948
+ has_quote	AUC 0.978	log loss 0.0889
+ days_in_pipeline	AUC 0.875	log loss 0.1708
+ first_response_hours	AUC 0.831	log loss 0.1926

Adding one leaky column to the honest model

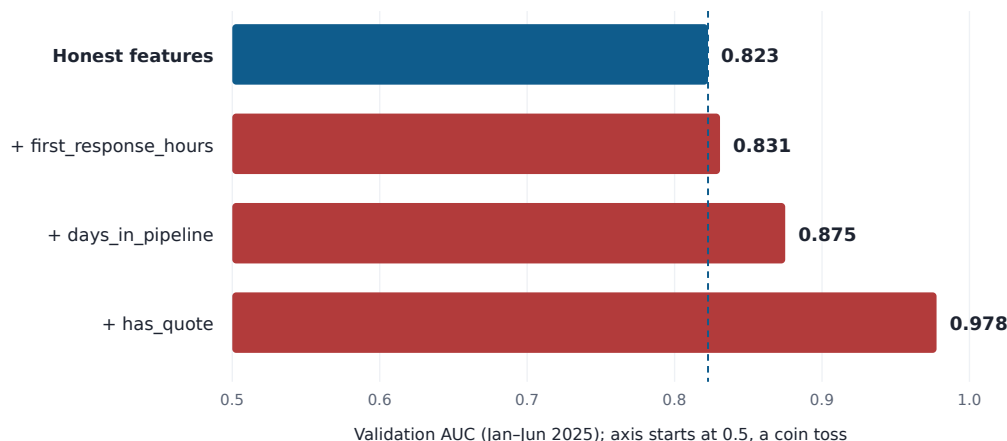


Figure 36.3 — What each leaky column does to the validation score. The bigger the jump, the easier the leak is to spot; the small one is the dangerous one.

Reading it.

- **has_quote** lifts AUC from 0.823 to **0.978**. A model that "knows" a quote was sent is nearly perfect, because Riverstone only quotes leads that are already going well. In production, at 24 hours, no lead has a quote yet: every value is 0, and the model collapses. A score that jumps this much should make you suspicious, not happy.
- **days_in_pipeline** lifts AUC to 0.875. Lost leads often close at exactly 90 days by default; won leads close whenever the order comes. The number is created by the outcome.
- **first_response_hours** lifts AUC only from 0.823 to **0.831**. That's the dangerous kind: plausible, modest, and readily explained away as "fast response wins deals". Part of it is real and already captured by `responded_24h`. The rest comes from two leaks. Reps respond faster to leads they already sense are good, so the model learns the reps' judgment (Chapter 35's warning). And the exact hours past 24 aren't known at the prediction moment, while leads never contacted at all, which are nearly always lost, are marked by a missing value that only exists once the 90 days are over.

Watch out: activity logs leak through timestamps. `activities.csv` has two times: when the call happened (`activity_at`) and when the rep logged it (`logged_at`). About 1 in 10 activities is logged days late, with the call's real time filled in. At 8 a.m. the next morning, a late-logged call is invisible. The code above counts only activities **logged** within 24 hours. Counting by `activity_at` would use calls the CRM didn't yet show. On this data the difference is small, but the habit matters: for every timestamp, ask which clock it is.

Contamination: target encoding on all the data

The second kind of leak happens in preprocessing. **Target encoding** replaces each category with the average target for that category, for example "the win rate of companies with this name". It's useful for columns with hundreds of values. Done carelessly, it's a leak:

```

wrong_rates = leads.groupby("company_name")["won"]
].mean() # uses ALL leads, including valid and test
leads["company_rate_wrong"] = leads["company_name"].map(wrong_rates)
train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]

print(leads["company_name"].nunique(), "distinct company names")
print(
    "with company win rate from ALL data: AUC %.3f  log loss %.4f"
    % score(cats, nums + ["company_rate_wrong"])
)

from sklearn.preprocessing import TargetEncoder

right = Pipeline(
    [
        (
            "prepare",
            ColumnTransformer(
                [
                    (
                        "cat",
                        Pipeline(
                            [
                                (
                                    "fill",
                                    SimpleImputer(
                                        strategy="constant", fill_value="Missing"
                                    ),
                                ),
                                ("onehot", OneHotEncoder(handle_unknown="ignore")),
                            ]
                        ),
                        cats,
                    ),
                    ("company", TargetEncoder(random_state=36), ["company_name"]),
                    (
                        "num",
                        Pipeline(
                            [
                                (
                                    "fill",
                                    SimpleImputer(
                                        strategy="median", add_indicator=True
                                    ),
                                ),
                                ("scale", StandardScaler()),
                            ]
                        ),
                        nums,
                    ),
                ],
            ),
        ),
        ("model", LogisticRegression(max_iter=1000)),
    ]
)
right.fit(train[cats + nums + ["company_name"]], train["won"])
p = right.predict_proba(valid[cats + nums + ["company_name"]])[:, 1]
print(
    "with TargetEncoder inside the pipeline: AUC %.3f  log loss %.4f"

```

```
% (roc_auc_score(valid["won"], p), log_loss(valid["won"], p))
)
```

```
300 distinct company names
with company win rate from ALL data: AUC 0.845 log loss 0.1849
with TargetEncoder inside the pipeline: AUC 0.823 log loss 0.1948
```

Reading it. Company names like "Lotus Kitchenware" carry no real signal in this data; they're generated at random. Yet encoding them with win rates computed on *all* leads lifts validation AUC from 0.823 to **0.845**, because each validation lead's own outcome went into its company's average. Done properly, with scikit-learn's `TargetEncoder` learning the rates from training rows only (and cross-fitting within them), the gain disappears: AUC is back at 0.823. The honest answer is that company name doesn't help.

How it works: `TargetEncoder` learns each category's target average (shrunk toward the overall average for rare categories) from the rows it's fitted on. When a pipeline is fitted, it calls the encoder's `fit_transform`, which uses internal **cross-fitting**: the training rows are split into folds, and each fold is encoded with averages learned from the other folds, so no training row's own outcome is used to encode that row. When the pipeline later transforms validation rows, it applies the averages learned from all the training rows. (Calling `fit` and then `transform` on the same rows skips the cross-fitting, which is why the encoder belongs inside the pipeline.)

A leakage checklist

Use this before trusting any model:

Check	Question to ask
Prediction moment	Is it written down, and is every feature available at that moment?
Timestamps	For every date, is it <i>when it happened</i> or <i>when it was recorded</i> ?
Outcome-driven fields	Was any field filled in by a process that only happens to some outcomes (quotes, approvals, cancellations)?
Missing values	Could "missing" mean "this hadn't happened yet"?
Duplicates	Does the same customer or enquiry appear in both training and test?
Preprocessing	Were scaling, filling, encoding, or feature selection fit on anything except training rows?
Target encodings and aggregates	Did any average include the row's own target, or future rows?
Too good to be true	Is the score far above what domain experts manage? Then assume a leak until proven otherwise

36.6 Feature engineering

A **feature** is a column the model learns from. **Feature engineering** turns raw columns into features that make patterns easier to learn. For the model families in this part of the book, especially linear models, it usually matters more than the choice of algorithm.

Numbers: transform, cap, and scale

`est_quantity` is the quantity the customer typed into the enquiry form. Look at it:

```
q = leads["est_quantity"]
print(q.describe(percentiles=[0.5, 0.9, 0.99]).round(0).to_string())
print("values above 50,000:", (q > 50_000).sum())
```

```
count      9161.0
mean       1120.0
std        18160.0
min         1.0
50%        130.0
90%        687.0
99%       3441.0
max      1214000.0
values above 50,000: 35
```

The median is 130 units and the 99th percentile about 3,400, but the maximum is over 1.2 million, and 35 values are above 50,000. Those extreme values are typing mistakes: someone entering "1200" in a field that already assumed "thousands", or adding zeros. Three steps handle this:

1. **Cap or remove impossible values.** Riverstone has never received an order anywhere near 50,000 units of one product, so values above that are set to missing, not kept.
2. **Transform skewed numbers.** The `log ( np.log1p`, which is $\ln(1 + x)$ and works for zeros) turns "each extra unit matters the same" into "each doubling matters the same". The difference between 10 and 100 units matters more than the difference between 2,010 and 2,100.
3. **Scale.** `StandardScaler` subtracts each column's mean and divides by its standard deviation, learned from the training rows. Chapter 35 showed why: gradient-based models train slowly, or badly, when features are on wildly different scales, and distance-based models are dominated by the biggest numbers.

When scaling doesn't matter. Decision trees and the tree ensembles built on them (random forests, gradient boosting, Chapter 37) split on one feature at a time at a threshold, so rescaling a feature doesn't change which rows fall on each side. Scaling them is harmless but unnecessary. Linear models, k-nearest neighbors, support vector machines, neural networks, k-means, and PCA all need it.

Categories: one-hot encoding

A model needs numbers, not the word "Referral". **One-hot encoding** creates one column per category, with a 1 in the column that matches and 0 elsewhere. `source` has 6 values, so it becomes 6 columns.

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
encoder.fit(train[["source"]])
sample = pd.DataFrame({"source": ["Referral", "Marketplace", "Newspaper ad"]})
print(encoder.get_feature_names_out())
print(encoder.transform(sample).toarray().astype(int))
```

```
[ 'source_Cold call' 'source_Marketplace' 'source_Partner'
  'source_Referral' 'source_Trade fair' 'source_Website']
[[0 0 0 1 0 0]
 [0 1 0 0 0 0]
 [0 0 0 0 0 0]]
```

How it works:

- The encoder learns the categories from the training rows. `get_feature_names_out` lists the new columns.
- A **Referral** lead becomes a 1 in the `source_Referral` column and 0 in the other five.
- **Newspaper ad** never appeared in training. With `handle_unknown="ignore"`, it becomes all zeros instead of crashing the model on the first unexpected value in production. New categories will appear; decide now how to treat them.
- The result is stored as a **sparse matrix**, which saves memory when most values are 0; `.toarray()` prints it as a normal grid.

Which encoding when:

Column type	Example	Encoding
Few categories, no order	source , segment	One-hot
Categories with a real order	company_size : 1-10 < 11-50 < 51-200 < 200+	One-hot, or ordinal numbers 1-4 if the order is meaningful to the model
Hundreds or thousands of categories	company name, postcode, product SKU	Target encoding inside the pipeline, grouping rare values into "Other", or leave it out
Free text	enquiry_text	Keyword flags (below), or TF-IDF (Chapter 41)

Dates and times

A timestamp is rarely useful raw; the model can't learn from "2024-10-14 11:32". Pull out the parts that could matter: **month** (festive season), **day of week** (weekend enquiries are often casual), **hour** (office-hours enquiries come from businesses), and **time since** something happened (days since the customer's last enquiry). Never use a date that's in the future relative to the prediction moment.

```

created = leads["created_at"]
date_parts = pd.DataFrame(
    {
        "month": created.dt.month,
        "weekday": created.dt.dayofweek,
        "hour": created.dt.hour,
        "won": leads["won"],
    }
)
by_month = (
    date_parts[date_parts["month"].isin([6, 7, 8, 9, 10, 11])]
    .groupby("month")["won"]
    .mean()
)
print("win rate by month (Jun-Nov):", (by_month * 100).round(1).to_dict())
hosp = leads[leads["segment"] == "Hospitality"]
festive = hosp[hosp["created_at"].dt.month.isin([8, 9, 10])]
print(
    f"hospitality win rate, Aug-Oct {hosp.loc[festive, 'won'].mean():.1%}"
    f" vs other months {hosp.loc[~festive, 'won'].mean():.1%}"
)

```

```

win rate by month (Jun-Nov): {6: 6.6, 7: 7.3, 8: 9.2, 9: 10.1, 10: 8.5, 11: 7.9}
hospitality win rate, Aug-Oct 8.9% vs other months 6.4%

```

Hospitality leads that arrive from August to October, as hotels and caterers prepare for the festive season, are won more often. A month feature alone gives a linear model an average monthly effect; the hospitality-in-festive-season effect is an **interaction** (two features together), which trees find on their own and linear models need built for them. Chapter 37 compares both.

Text: start with flags

`enquiry_text` is short free text. A good first step is **keyword flags**: 1 if the text mentions a strong buying signal ("bulk", "tender", "urgent"), 1 if it signals browsing ("price list", "sample only").

`add_features` built both:

```

flag_rates = leads.groupby(["text_strong", "text_weak"])["won"].agg(["count", "mean"])
flag_rates["mean"] = (flag_rates["mean"] * 100).round(1)
print(flag_rates.rename(columns={"count": "leads", "mean": "win rate %"}).to_string())
print(
    "free email (gmail) win rate: %.1f%% vs business email: %.1f%%"
    % tuple(leads.groupby("free_email")["won"].mean()[[1, 0]] * 100)
)

```

	text_strong	text_weak	leads	win rate %
0	0	0	5125	7.3
		1	3219	5.1
1	0	0	2357	12.1
free email (gmail) win rate: 4.8% vs business email: 10.0%				

Flags are crude but transparent, and a sales manager can check them. Chapter 41 turns text into features properly with TF-IDF and embeddings. The email domain is another text feature: a business domain (`@sharmahardware.in`) signals a real company.

Aggregates from activity logs

Many strong features are **aggregates**: counts, sums, averages, or recency computed from a related table. Here, `activities_24h` counts activities logged within 24 hours of arrival. Other useful aggregates for lead scoring would be the number of earlier enquiries from the same company, or days since the company's last order.

The rule for aggregates: compute them **as of the prediction moment**, for each row separately. "Total activities on this lead" (counting the whole 90 days) is target leakage; "activities logged in the first 24 hours" isn't.

36.7 Missing values in machine learning

Most scikit-learn models refuse to train on missing values (`ValueError: Input X contains NaN`). You have three choices: drop rows, drop columns, or **impute** (fill in) values. Before choosing, look at *which* values are missing and whether missingness means something:

```
missing = (
    leads[["segment", "company_size", "est_quantity", "log_quantity"]].isna().mean()
)
print("share missing:", (missing * 100).round(1).to_dict())
q_missing = leads["est_quantity"].isna()
print(
    f"win rate when quantity is missing {leads.loc[q_missing, 'won'].mean():.2%}, "
    f"when given {leads.loc[~q_missing, 'won'].mean():.2%}"
)
print(
    "quantity missing, by source:",
    (leads.groupby("source")["est_quantity"].apply(lambda s: s.isna().mean()) * 100)
    .round(1)
    .to_dict(),
)
```

```
share missing: {'segment': 5.2, 'company_size': 20.8, 'est_quantity': 14.4, 'log_quantity': 14.7}
win rate when quantity is missing 5.71%, when given 8.06%
quantity missing, by source: {'Cold call': 7.0, 'Marketplace': 28.4, 'Partner': 6.0, 'Referral':
5.1, 'Trade fair': 7.0, 'Website': 6.5}
```

Reading it. Quantity is missing for about one lead in seven, and those leads are won less often (5.71% against 8.06%). The reason shows in the last line: marketplace enquiry forms let buyers skip the quantity box, and marketplace leads convert poorly. **Missingness is information.** Filling the blanks with the median and moving on would throw that information away.

The standard approach has two parts:

1. **Impute** a reasonable value so the model can run: the median for numbers, a "Missing" category for categories.
2. **Add a missing indicator**, a 0/1 column saying "this was blank", so the model can learn whether blanks matter. `SimpleImputer(add_indicator=True)` does both at once.

Strategy	When it fits	Risk
Drop rows with any missing value	A handful of rows, missing at random	Loses data; biased if missingness is related to the target
Drop the column	Mostly empty, or not useful	Loses a feature
Median (numbers) or most frequent (categories)	Few blanks, no pattern	Hides informative missingness
"Missing" as its own category	Categorical columns	None serious; usually a good default
Impute + missing indicator	Missingness may carry signal	Adds columns
Model-based imputation (<code>KNNImputer</code> , <code>IterativeImputer</code>)	Many related columns	Slower; more complex; rarely worth it for lead scoring

Watch out: learn the fill values from training rows only. The median used to fill blanks in validation and test rows must be the **training** median. Computing it on the whole dataset is train-test contamination, the same mistake as the target-encoding leak, only smaller. Inside a pipeline, this happens automatically.

Gradient-boosting libraries such as XGBoost and LightGBM, and scikit-learn's `HistGradientBoostingClassifier` , handle missing values natively by learning which branch blanks should follow (Chapter 37). An indicator is still a good habit when blanks might mean something.

36.8 scikit-learn pipelines

The problem pipelines solve

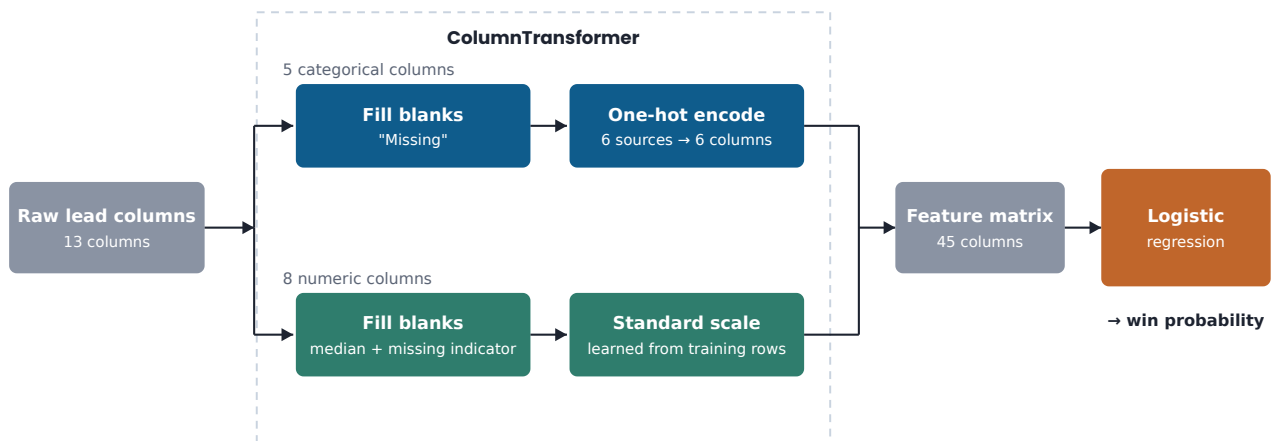
Every learned preprocessing step (the median, the scaler's mean and standard deviation, the categories, the target encoding) must be:

- **learned** from the training rows only,
- **applied** unchanged to validation, test, and new production rows,
- **re-learned** inside each fold during cross-validation.

Doing that by hand means writing the same steps three times and never making a mistake. A **pipeline** bundles the steps and the model into one object with one `fit` and one `predict` , so it can't go wrong.

The pieces

- **Pipeline** chains steps in order: each step's output feeds the next, and the last step is the model.
- **ColumnTransformer** applies different steps to different columns (one recipe for categories, another for numbers) and glues the results side by side.
- **Transformers** (`SimpleImputer` , `OneHotEncoder` , `StandardScaler` , `TargetEncoder`) have `fit` (learn from data) and `transform` (apply what was learned).



`fit(train)` learns every step from training rows · `predict_proba(new rows)` applies them unchanged

Figure 36.4 – The lead-scoring pipeline. `fit` learns every step from training rows; `predict_proba` applies them, unchanged, to new rows.

The `make_model` function from section 36.4 is exactly this. Look inside the fitted version:

```

pipeline = make_model(cats, nums).fit(train[cats + nums], train["won"])
prepare = pipeline.named_steps["prepare"]
names = prepare.get_feature_names_out()
print(len(cats) + len(nums), "raw columns ->", len(names), "model features")
print(names[:4], "...", names[-5:])
print(
    "median log quantity learned from training rows:",
    prepare.named_transformers_["num"].named_steps["fill"].statistics_[0].round(3),
)

one_new_lead = pd.DataFrame(
    [
        {
            "source": "Referral",
            "segment": "Hospitality",
            "city_clean": "Pune",
            "company_size": np.nan,
            "product_interest": "Kitchen",
            "log_quantity": np.loglp(400),
            "website_visits": 2,
            "free_email": 0,
            "text_strong": 1,
            "text_weak": 0,
            "inside_desk": 0,
            "activities_24h": 1,
            "responded_24h": 1,
        }
    ]
)
print(
    f"win probability for this lead: {pipeline.predict_proba(one_new_lead)[0, 1]:.1%}"
)

```

```

13 raw columns -> 45 model features
['cat_source_Cold call' 'cat_source_Marketplace' 'cat_source_Partner'
 'cat_source_Referral'] ... ['num_text_weak' 'num_inside_desk' 'num_activities_24h'
 'num_responded_24h' 'num_missingindicator_log_quantity']
median log quantity learned from training rows: 4.875
win probability for this lead: 44.5%

```

How it works:

- `named_steps` and `named_transformers_` let you reach inside a fitted pipeline to check what each step learned. The imputer's `statistics_` holds the training medians.
- The 13 raw columns become 45 features: the one-hot columns for the 5 categorical columns (including a `Missing` column where blanks exist), the 8 scaled numbers, and one missing indicator, for `log_quantity`, the only numeric column with blanks in training.
- Scoring a brand-new lead, even one with a blank company size, needs one line. The same object that was validated is the one that goes to production (Chapter 52 covers deploying it).

Proof that it prevents contamination

Section 36.4 passed the whole pipeline to `cross_val_score`, so every median and scaler was re-learned inside each fold. The common mistake is to prepare the data once, then cross-validate the model alone. For medians and scaling, the damage is usually tiny, as here; for target encoding and feature selection, it's large. The pipeline makes the question irrelevant.

```

from sklearn.base import clone

X_train = train[cats + nums + ["company_name"]]
folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=36)

# Wrong: target-encode company names on all training rows, then cross-validate
prepared_once = X_train.copy()
prepared_once["company_rate"] = train["company_name"].map(
    train.groupby("company_name")["won"].mean()
)
wrong = cross_val_score(
    make_model(cats, nums + ["company_rate"]),
    prepared_once[cats + nums + ["company_rate"]],
    train["won"],
    cv=folds,
    scoring="roc_auc",
)

# Right: the encoder is inside the pipeline, so each fold learns its own rates
honest = cross_val_score(
    clone(right), X_train, train["won"], cv=folds, scoring="roc_auc"
)
print("encode first, then cross-validate: mean AUC", wrong.mean().round(3))
print("encoder inside the pipeline:          mean AUC", honest.mean().round(3))

```

```

encode first, then cross-validate: mean AUC 0.82
encoder inside the pipeline:      mean AUC 0.78

```

The wrong version reports a cross-validation AUC several points higher than the honest one. Nothing about the model changed; only *where* the encoding was learned. `clone` makes a fresh, unfitted copy of the pipeline, so nothing from the earlier `fit` carries over.

36.9 Baselines, and the final test

Always beat something simple first

A model score means nothing on its own. Before celebrating an AUC of 0.8, compare it with things that need no machine learning:

1. **Base rate:** predict the training win rate for every lead. It can't rank anything (AUC 0.5), but its log loss is the benchmark for probabilities (Chapter 35).
2. **A rule of thumb:** what the sales team does today, "call referrals, trade-fair contacts, and partner leads first". Score it as 1 for those sources and 0 for the rest.
3. **The simplest real model:** logistic regression on a handful of clean features, the pipeline you've built.

Chapter 37's more complex models must beat number 3 by a margin worth their extra complexity.

```
def baselines(fit_rows, eval_rows):
    y = eval_rows["won"]
    base = np.full(len(eval_rows), fit_rows["won"].mean())
    rule = eval_rows["source"].isin(["Referral", "Trade fair", "Partner"]).astype(int)
    model = make_model(cats, nums).fit(fit_rows[cats + nums], fit_rows["won"])
    p = model.predict_proba(eval_rows[cats + nums])[:, 1]
    rate = fit_rows["won"].mean()
    print(f" base rate {rate:.2%}:   AUC 0.500   log loss {log_loss(y, base):.4f}")
    rule_auc = roc_auc_score(y, rule)
    print(f" rule (3 sources):      AUC {rule_auc:.3f}   log loss  n/a (not a probability)")
    model_auc, model_ll = roc_auc_score(y, p), log_loss(y, p)
    print(f" logistic regression: AUC {model_auc:.3f}   log loss {model_ll:.4f}")
    return p

print("VALIDATION (trained on 2023-2024):")
p_valid = baselines(train, valid)
print(
    f" average predicted win rate {p_valid.mean():.2%} vs actual {valid['won'].mean():.2%}"
)
```

```
VALIDATION (trained on 2023-2024):
base rate 7.90%:   AUC 0.500   log loss 0.2435
rule (3 sources):   AUC 0.680   log loss  n/a (not a probability)
logistic regression: AUC 0.823   log loss 0.1948
average predicted win rate 6.90% vs actual 6.56%
```

Reading it. On validation, the logistic regression ranks leads much better than the rule (AUC 0.823 against 0.680) and its probabilities beat the base rate clearly (log loss 0.1948 against 0.2435). It also passes a simple sanity check on the probabilities: it predicts an average win rate of 6.90% for the first

half of 2025, close to the actual 6.56%. That's slightly high, because the model learned from 2023–2024, when marketplace leads converted better. Chapter 39 calls this **calibration** and shows how to check it properly.

Back to the random split

Section 36.3 warned that scores from different splits can't be compared. Here's the evidence: the same pipeline, evaluated on a random slice of the same number of leads, drawn from all periods before July 2025:

```
from sklearn.model_selection import train_test_split

before_test = leads[leads["created_at"] < "2025-07-01"]
rand_train, rand_valid = train_test_split(
    before_test, test_size=len(valid), random_state=36, stratify=before_test["won"]
)
print("time split (valid = Jan-Jun 2025):  AUC %.3f  log loss %.4f" % score(cats, nums))
print(
    "random split (same number of rows): AUC %.3f  log loss %.4f"
    % score(cats, nums, rand_train, rand_valid)
)
```

```
time split (valid = Jan-Jun 2025):  AUC 0.823  log loss 0.1948
random split (same number of rows): AUC 0.791  log loss 0.2269
```

The random split scores lower on both measures. That isn't because it's more honest; its validation rows are a different mix (fewer of 2025's clearly weak marketplace leads, and a higher win rate). Neither number is "the" score. The time split's number answers the question Anita cares about, how the model does on later leads, so it's the one to report.

The one-time test

All the choices are made: features, encodings, split, model. Now, once, retrain on training plus validation (more recent data helps) and score the locked test set:

```
final_fit = pd.concat([train, valid])
print("TEST (trained on 2023 to June 2025, scored once on July to 2 October 2025):")
p_test = baselines(final_fit, test)
```

```
TEST (trained on 2023 to June 2025, scored once on July to 2 October 2025):
base rate 7.59%:  AUC 0.500  log loss 0.2983
rule (3 sources):  AUC 0.727  log loss  n/a (not a probability)
logistic regression: AUC 0.838  log loss 0.2369
```

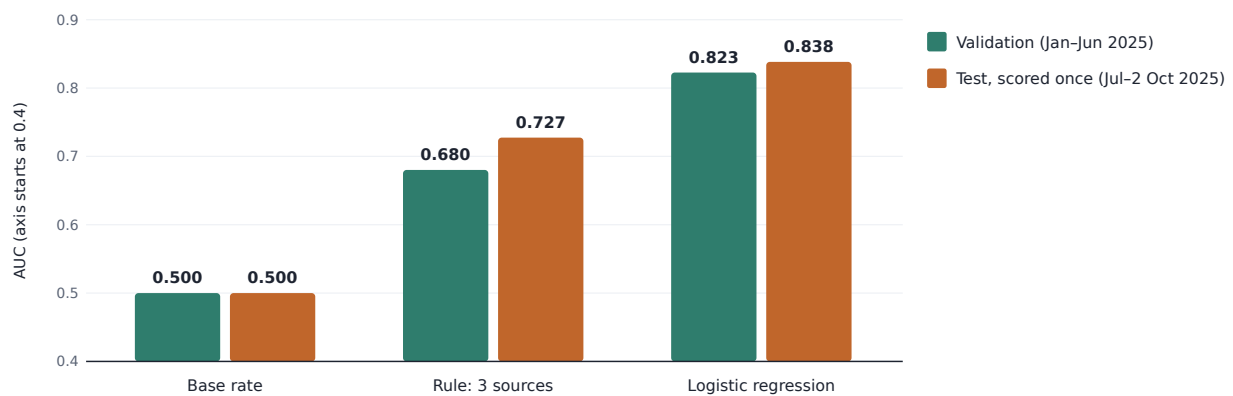


Figure 36.5 – Baselines against the model. The rule helps; the model helps more, on the validation set used for choices and the test set used once.

Reading it. On the untouched test set, the model scores an AUC of **0.838** and a log loss of **0.2369**, against the rule's 0.727 and the base rate's 0.2983. The test score is a little higher than validation, not lower: the test period's win rate (8.78%) and mix differ from the first half of 2025, and a few hundred leads with about a hundred wins give a noisy estimate. The honest summary isn't "0.838"; it's "around 0.8, clearly better than the current rule, on leads the model had never seen."

What to tell Anita. "Using only what's in the CRM the morning after a lead arrives, the model ranks leads clearly better than calling referral, trade-fair, and partner leads first. If a rep calls leads in the model's order, the leads they reach first are much more likely to be won. Before we rely on it, I want to run it quietly alongside the team for a month and compare, and Chapter 39's cost analysis will tell us how many leads to call." (That last sentence is a promise the next three chapters keep.)

Common mistakes and how to spot them

Mistake	Symptom	Fix
No prediction moment defined	Features from after the decision sneak in	Write down <i>when</i> the prediction is made; test every feature against it
Using leads whose outcome isn't settled	Recent periods look terrible	Keep only rows whose outcome window has fully passed
Random split for a problem that runs forward in time	Model looks fine in testing, disappoints in production	Split by time, in the direction it will be used
Tuning on the test set	Test score keeps improving as you try things	Use validation or cross-validation for choices; touch test once
Duplicates across train and test	Suspiciously good scores	Deduplicate first; for repeat customers, split by customer (group split)
Outcome-driven columns (quotes, closing dates, final status)	AUC near 0.95–1.0	Remove anything filled in after, or because of, the outcome

Mistake	Symptom	Fix
Preprocessing fit on all data	Validation score slightly optimistic	Put every learned step in a pipeline
Target encoding outside cross-validation	Big gains from high-cardinality columns	Use <code>TargetEncoder</code> inside the pipeline
Filling blanks and discarding the fact they were blank	Loses signal from informative missingness	<code>SimpleImputer(add_indicator=True)</code>
Keeping impossible values	Huge coefficients, convergence warnings, odd predictions	Cap or set to missing based on domain limits; log-transform skewed numbers
Failing on new categories in production	Error on the first unseen value	<code>OneHotEncoder(handle_unknown="ignore")</code>
No baseline	"0.8 AUC" with no idea whether that's good	Report base rate, the current rule, and a simple model together
Comparing scores from different splits	Arguments about which model is "better"	Compare models only on the same rows
Treating one validation score as exact	Chasing changes of 0.005	Check cross-validation spread; changes smaller than it are noise

In the real world: Vikram's 98% model

In March 2026, Vikram Singh, Riverstone's Sales Manager, shares a notebook from a freelance analyst he has hired for a month. The headline: *"Lead conversion model: 98% accuracy, AUC 0.99."* He wants to roll it out to the sales team the next Monday and asks Meera Iyer to "sanity check the numbers".

Meera doesn't start with the algorithm. She opens the list of features and asks the one question from section 36.5 of each: *would this be in the CRM the morning after the lead arrives?*

The feature list has 31 columns. Twenty-seven pass. Four don't:

- `stage_reached` , the furthest pipeline stage. It is the outcome, in slightly different words.
- `quote_value` , filled in only when a quote is sent.
- `total_activities` , counting every call and email over the lead's whole life. Won leads get many follow-up calls because they're being won.
- `lost_reason` , blank for every won lead. The model learned that "blank means won".

Next, the accuracy. At Riverstone's 7.7% win rate, predicting "lost" for every lead is 92.3% accurate. 98% is better, but not the miracle it sounds like.

Then the split. The analyst used a random 80/20 split across 2023–2025, and included the leads created in the last quarter of 2025, whose outcomes weren't settled. Many of those "lost" leads were still being worked.

Meera rebuilds the evaluation in an afternoon: the four columns removed, recent leads excluded, trained on 2023–2024, validated on the first half of 2025. The model's AUC falls from 0.99 to about 0.82, roughly the same as the simple pipeline from this chapter, and it still clearly beats the team's "referrals first" rule.

Her message to Vikram:

"The model is useful, but not for the reason the notebook says. Four of its columns are only filled in after a lead is won or lost, so on Monday morning they'd all be empty and the scores would be meaningless. With those removed and the model tested on later leads, it ranks leads at about 0.82 AUC, which is still clearly better than calling referrals first. I'd suggest we pilot the fixed version with Neha's leads for a month, and ask the analyst to add a written prediction-moment rule to the notebook."

Vikram's reply is one line: "Glad we didn't send the 98% slide to Anita."

The fixed model wasn't worse than the original. It was the same model, measured without the leaks.

Tools

- **scikit-learn:** every example was tested on 1.8.0; the current release at the time of writing is 1.9.1, so if a number differs slightly on your machine, check the version first. The pieces used are `Pipeline`, `ColumnTransformer`, `SimpleImputer`, `OneHotEncoder`, `StandardScaler`, `TargetEncoder` (available since version 1.3), `StratifiedKFold`, `TimeSeriesSplit`, `cross_val_score`, `LogisticRegression`, and the metrics `roc_auc_score` and `log_loss`. The official user guide's pages on cross-validation, pipelines, and "common pitfalls" (which covers leakage) are worth reading in full.
 - **pandas** (3.0.2) and **NumPy** (2.4.4) for cleaning and feature building.
 - **Python** 3.12.3. Every example was run on 17 September 2026; the APIs used have been stable since scikit-learn 1.3, when `TargetEncoder` was added.
 - **In SQL:** the deduplication, the 90-day maturity filter, and the "activities logged within 24 hours" aggregate can all be done in the database with Chapter 13's patterns, which is often where production feature pipelines start (Chapter 47 on data pipelines).
 - **Beyond this chapter:** `imbalanced-learn` works with pipelines for resampling (Chapter 39), and `feature-engine` and `category_encoders` offer more transformers. Use them once the basics here feel routine.
 - **Companion files:** `companion/generate_riverstone_crm.py` rebuilds the CRM export (seed 20236) into `companion/crm/`. Run the chapter's code from that folder. The data spec is `planning/data/riverstone-crm.md`.
-

The project: a leakage-free feature pipeline for Riverstone lead scoring

Goal: a tested, documented pipeline that turns raw CRM exports into model-ready features for lead scoring, with an honest evaluation against baselines. Chapters 37 and 39 build on it, so keep it tidy.

Option A: your own data. Any prediction problem at work with a clear outcome and a time dimension: invoices paid late, customers who don't reorder, support tickets that escalate. Remove personal and confidential fields first, and check you're allowed to use the data.

Option B: Riverstone. The CRM export in `companion/crm/`.

Steps:

1. **Frame it.** Write the five-row framing table from section 36.1: unit, target, prediction moment, allowed information, success measure.
2. **Clean to one row per unit.** Deduplicate, fix category spellings, and exclude rows whose outcome window hasn't passed. Record how many rows each step removes.
3. **Split.** Make a time-based train, validation, and test split. Print the size and target rate of each.
4. **Audit every column for leakage.** For each raw column, write one line: known at the prediction moment, yes or no, and why. Include at least one timestamp check.
5. **Engineer at least six features**, including one log-transformed number, one date part, one text flag, and one aggregate from `activities.csv` computed as of the prediction moment.
6. **Handle missing values** with imputation and missing indicators, and show the target rate for blank versus filled values of one column.
7. **Build the pipeline** with `ColumnTransformer`, and cross-validate it with stratified 5-fold. Report the mean and standard deviation.
8. **Demonstrate one leak.** Add one leaky feature, report the inflated score, remove it, and explain in two sentences why it leaks.
9. **Compare with baselines** on validation: base rate, a rule of thumb, and your pipeline. Then score the test set once.
10. **Write a one-page summary** for a non-technical manager: what the model does, how well, compared with what, and one risk.

Stretch goals:

- Add a `FunctionTransformer` or a small custom transformer class so that `add_features` itself runs inside the pipeline, and the pipeline takes raw CRM columns directly.
- Replace the three-source rule with the best single-feature rule you can find on training data, and check whether it still holds on validation.
- Add "number of earlier enquiries from the same email" as an aggregate, computed strictly from leads that arrived before each lead.
- Save the fitted pipeline with `joblib.dump` and load it in a fresh Python session to score five new leads (Chapter 52 covers this properly).

You've got it when...

- ☐ I can write the target, unit, prediction moment, and success measure for a business request before touching data.
 - ☐ I know why leads, loans, or subscriptions whose outcome window hasn't passed must be excluded.
 - ☐ I can explain the jobs of training, validation, and test sets, and why the test set is used once.
 - ☐ I choose a time-based split when the model will predict forward in time, and I compare models only on the same split.
 - ☐ I can run stratified and time-series cross-validation and interpret the spread of the scores.
 - ☐ For any feature, I ask whether it would be known at the prediction moment, and I check which clock each timestamp uses.
 - ☐ I can explain target leakage and train-test contamination, with an example of each.
 - ☐ I can engineer features from numbers (cap, log, scale), categories (one-hot, target encoding), dates, text flags, and aggregates.
 - ☐ I check whether missingness predicts the target, and I use imputation with missing indicators.
 - ☐ I can build a `Pipeline` with a `ColumnTransformer`, inspect what it learned, and score a new row with it.
 - ☐ I always report a model next to a base rate and a rule-of-thumb baseline.
-

Recap

- **Frame first:** unit, **target**, **prediction moment**, allowed information, and success measure. Most leakage is prevented here.
- Exclude rows whose outcome isn't settled (**censoring**).
- **Training** data teaches, **validation** data guides choices, the **test** set is used once. Split by **time** when the model predicts forward; scores from different splits can't be compared.
- **Cross-validation** gives several scores; their spread is your uncertainty. Use **stratified** folds for classification and **time-series splits** when order matters.
- **Data leakage** is information the model won't have at prediction time. **Target leakage** comes from columns created after or because of the outcome; **train-test contamination** comes from preprocessing learned on evaluation rows. Ask: *would this be known at the prediction moment?*
- Big leaks look spectacular; small leaks look plausible. Both are wrong.
- **Feature engineering:** cap impossible values, **log-transform** skewed numbers, **scale** for non-tree models, **one-hot encode** categories, **target encode** high-cardinality columns inside the pipeline, extract **date parts**, add **text flags**, and build **aggregates** as of the prediction moment.
- **Missingness can be information:** impute, and add **missing indicators**.
- A **pipeline** learns every step from training rows and applies it unchanged everywhere else, including inside cross-validation.

- Always compare with **baselines**: the base rate, a rule of thumb, and a simple model.

Practice exercises

Code exercises run from `companion/crm/` after the chapter's code (they use `leads`, `train`, `valid`, `test`, `cats`, `nums`, `make_model`, and `score`). Predict each answer before running it.

Warm-up

1. For each request, write the target and the prediction moment: (a) "Which customers will stop ordering?" (b) "Which invoices will be paid late?" (c) "How many Industrial Crates will we sell next month?" Say which is classification and which is regression.
2. Which of these columns could be used to score a lead 24 hours after it arrives? (a) `source`; (b) `status`; (c) the number of website visits before the enquiry; (d) the date the lead was marked Qualified; (e) whether the lead was assigned to the inside sales desk.
3. A colleague's lead model uses a random split and scores AUC 0.80 on its test set; yours uses a time split and scores 0.78. Can you conclude theirs is better? Explain in two sentences.
4. `company_size` has four bands: 1-10, 11-50, 51-200, 200+. Give one reason to one-hot encode it and one reason to encode it as the numbers 1 to 4.

Core

5. Count how many 2025 leads (before the 2 October cutoff) come from each `source`, and the win rate for each. Which source's win rate changed most compared with 2023–2024?
6. Add a `weekday` feature (0 = Monday) to the model as a categorical column. Report validation AUC and log loss before and after. Is the change bigger than the cross-validation spread you saw in section 36.4?
7. Count activities logged within 24 hours using `activity_at` instead of `logged_at`, rebuild the feature, and compare validation AUC with the honest version. How big is the difference, and why should you still use `logged_at`?
8. Train the model without `responded_24h` and `activities_24h`. How much validation AUC do the two process features add? Is there any reason to be cautious about them even though they pass the prediction-moment test?
9. Build a pipeline where the numeric imputer does **not** add missing indicators. Compare validation log loss with the original. Explain the result using what section 36.7 found about quantity.
10. Run stratified 5-fold cross-validation of the full feature set (`cats` and `nums`) on the training rows. Report each fold's AUC, the mean, and the standard deviation.

Stretch

11. Write a function `leak_check(column)` that adds one column to the honest features and reports the change in validation AUC. Run it on `days_in_pipeline`, `has_quote`, and `lead_id`. What change would make you investigate a column?

- l2. Some companies send several genuine enquiries over three years. Check how many distinct `email_norm` values appear in both `train` and `test`. Why could this matter, and what kind of split would prevent it?
- l3. Replace logistic regression in `make_model` with `HistGradientBoostingClassifier(random_state=36)`. Does it beat logistic regression on validation? (Chapter 37 explains why the answer can go either way.)

Think about it

- l4. Your model scores 0.97 AUC on validation for predicting which support tickets will escalate. Name three things you'd check before telling anyone.
- l5. The sales team starts calling leads in the model's order. Six months later you retrain on the new data. What has changed about the data that could make the new model misleading? (Hint: who got called, and when.)
- l6. A manager asks you to "just use all the columns; the model will figure out which ones matter". Write a two-sentence reply.

Key terms

supervised learning · binary classification · regression · target · unit of analysis · prediction moment · censoring · deduplication · training set · validation set · test set · random split · time-based split · data drift · cross-validation · fold · k-fold · stratified k-fold · time series split · ROC-AUC · data leakage · target leakage · train-test contamination · outcome-driven field · feature · feature engineering · log transform · capping · scaling (standardization) · one-hot encoding · sparse matrix · ordinal encoding · target encoding · cross-fitting · high cardinality · date parts · interaction · keyword flag · aggregate feature · missing value · imputation · missing indicator · informative missingness · pipeline · `ColumnTransformer` · `transformer (fit / transform)` · baseline · rule-of-thumb baseline · calibration

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 37, Supervised Learning Algorithms**, swaps different models into this pipeline, from regularized logistic regression to gradient boosting, and asks whether they beat the baseline by enough to matter.
- **Chapter 38, Unsupervised Learning**, reuses scaling and encoding for clustering, where there's no target to leak.
- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, takes this chapter's validation predictions and turns them into a call list with a cost-based threshold, checks calibration, and explains which features drive the scores.
- **Chapter 40, Time Series & Forecasting**, extends time-based splitting into backtesting.

- **Chapter 41, NLP Foundations**, replaces keyword flags with TF-IDF features for text.
- **Chapter 44, Capstone**, runs this whole workflow on a new question from start to presentation.
- **Chapters 47 and 52** move feature pipelines into scheduled data pipelines and deployed models.
- **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers leakage, splits, cross-validation, encoding, and missing values with graded answers. Leakage questions are among the most common in data science interviews.

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. (a) Target: the customer places no order in the next 90 days, 1 or 0. Prediction moment: the first day of each month, for every active customer, using order history up to that day. Classification. (b) Target: the invoice is paid more than N days after its due date, 1 or 0. Prediction moment: the day the invoice is issued. Classification. (c) Target: units of Industrial Crates sold next month, a number. Prediction moment: the last day of the current month. Regression (and a time series problem, Chapter 40). The common gap is the prediction moment: without it, "will stop ordering" can quietly use next month's orders.

2. (a) Yes, it's on the enquiry form. (b) No, `status` is the outcome. (c) Yes, if it counts visits *before* the enquiry. (d) No: qualification happens days after arrival and mostly to leads that go on to be won. (e) Yes: leads are assigned on arrival. Check how the assignment is made, though: if the desk is given leads a rep has already judged weak, the assignment carries that judgment, like the response-time leak.

3. No. Their score is measured on different rows, with a different mix of leads (a random slice of all years, not the most recent months), and section 36.3 showed the mix alone can move AUC by a few points. Score both models on the same time-based validation set before comparing.

4. One-hot encoding makes no assumption about the gaps between bands: the model can learn that 11-50 and 51-200 behave alike while 200+ is very different. Encoding as 1 to 4 uses one column instead of four and tells the model the bands are ordered, which helps when data is scarce and the effect really does rise steadily with size. A tree-based model can use the ordered numbers well; a linear model assumes each step up is worth the same.

5.

```

period = np.where(leads["created_at"] < "2025-01-01", "2023-24", "2025")
table = leads.pivot_table(
    index="source", columns=period, values="won", aggfunc=["count", "mean"]
)
table.columns = [f"{a}_{b}" for a, b in table.columns]
table["change_pts"] = (table["mean_2025"] - table["mean_2023-24"]) * 100
print(
    table.assign(**{c: table[c] * 100 for c in ["mean_2023-24", "mean_2025"]})
    .round(2)
    .to_string()
)

```

source	count_2023-24	count_2025	mean_2023-24	mean_2025	change_pts
Cold call	647	206	8.50	8.74	0.24
Marketplace	2381	1516	2.98	1.52	-1.46
Partner	559	170	10.91	12.94	2.03
Referral	838	290	19.33	23.79	4.46
Trade fair	845	411	13.37	16.06	2.69
Website	2021	817	5.64	6.36	0.72

The biggest change is the marketplace, whose win rate fell by about 1.4 percentage points while its lead count in 2025 grew to the largest of any source. A drop from about 3% to 1.5% is small in points but halves the rate. Referral, trade-fair, and partner leads rose by 2 to 4.5 points; with only a few hundred 2025 leads each, changes of that size can come from chance alone (Chapter 22 shows how to check).

6.

```

leads["weekday"] = leads["created_at"].dt.dayofweek.astype(str)
train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]
print("without weekday: AUC %.3f log loss %.4f" % score(cats, nums))
print("with weekday: AUC %.3f log loss %.4f" % score(cats + ["weekday"], nums))

```

```

without weekday: AUC 0.823 log loss 0.1948
with weekday: AUC 0.819 log loss 0.1965

```

Both scores get slightly worse (AUC 0.819, log loss 0.1965), a change within the fold-to-fold spread you saw in section 36.4. The generator planted no weekday effect, so this is the expected honest result: a feature that sounds reasonable and adds nothing. Keeping it would add seven columns of noise. (`astype(str)` makes the encoder treat days as categories, not as numbers 0 to 6.)

7.

```

by_happened = activities[
    activities["activity_at"] <= activities["lead_id"].map(deadline)
]
leads["activities_24h_wrong"] = (
    leads["lead_id"].map(by_happened.groupby("lead_id").size()).fillna(0)
)
train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]
wrong_nums = [c if c != "activities_24h" else "activities_24h_wrong" for c in nums]
print("logged_at (honest):      AUC %.3f  log loss %.4f" % score(cats, nums))
print("activity_at (leaky):     AUC %.3f  log loss %.4f" % score(cats, wrong_nums))
print(
    "leads whose count differs:",
    (leads["activities_24h"] != leads["activities_24h_wrong"]).sum(),
)

```

```

logged_at (honest):      AUC 0.823  log loss 0.1948
activity_at (leaky):     AUC 0.825  log loss 0.1929
leads whose count differs: 781

```

The leaky version scores slightly better (AUC 0.825 against 0.823), and 781 leads get a different count. On this data the gap is small. Use `logged_at` anyway: the honest feature is the one the CRM can actually show at 8 a.m., and in a team where logging late is common, the gap would be larger and would grow silently. Evaluation must mirror production exactly, not approximately.

8.

```

without_process = [c for c in nums if c not in ("responded_24h", "activities_24h")]
print("with process features:    AUC %.3f  log loss %.4f" % score(cats, nums))
print(
    "without process features: AUC %.3f  log loss %.4f" % score(cats, without_process)
)

```

```

with process features:    AUC 0.823  log loss 0.1948
without process features: AUC 0.814  log loss 0.1983

```

They add 0.009 AUC (0.823 against 0.814). Be cautious for the reason from section 36.5: a quick response partly reflects the rep's own judgment of the lead. They pass the prediction-moment test, but once the model's scores drive who gets called, reps will respond fastest to high-scoring leads, and the feature will start measuring the model's own past output (a feedback loop, exercise 15). Many teams leave process features out of the first model for that reason.

9.

```
def make_model_no_indicator(categorical, numeric):
    prepare = ColumnTransformer(
        [
            (
                "cat",
                Pipeline(
                    [
                        (
                            "fill",
                            SimpleImputer(strategy="constant", fill_value="Missing"),
                        ),
                        ("onehot", OneHotEncoder(handle_unknown="ignore")),
                    ]
                ),
                categorical,
            ),
            (
                "num",
                Pipeline(
                    [
                        ("fill", SimpleImputer(strategy="median")),
                        ("scale", StandardScaler()),
                    ]
                ),
                numeric,
            ),
        ]
    )
    return Pipeline(
        [("prepare", prepare), ("model", LogisticRegression(max_iter=1000))]
    )

m = make_model_no_indicator(cats, nums).fit(train[cats + nums], train["won"])
p = m.predict_proba(valid[cats + nums])[:, 1]
print(
    "without indicators: AUC %.3f log loss %.4f"
    % (roc_auc_score(valid["won"], p), log_loss(valid["won"], p))
)
print("with indicators:    AUC %.3f log loss %.4f" % score(cats, nums))
```

```
without indicators: AUC 0.822 log loss 0.1952
with indicators:    AUC 0.823 log loss 0.1948
```

Without the indicator, a blank quantity becomes the median, so the model treats a marketplace lead that skipped the box like an ordinary lead with a typical order size. The indicator lets it learn that blanks win less often (section 36.7). Here the effect is small, because `source` already tells the model a lead came from the marketplace and so carries much of the same information; with fewer overlapping features, the indicator would matter more.

10.

```

folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=36)
cv_full = cross_val_score(
    make_model(cats, nums),
    train[cats + nums],
    train["won"],
    cv=folds,
    scoring="roc_auc",
)
print(
    "fold AUCs:",
    cv_full.round(3),
    " mean",
    cv_full.mean().round(3),
    " sd",
    cv_full.std().round(3),
)

```

```
fold AUCs: [0.779 0.78 0.801 0.761 0.784] mean 0.781 sd 0.013
```

With all features, the mean rises from 0.741 to 0.781, and the folds now spread more (a standard deviation of 0.013). Notice that the cross-validation mean on 2023–2024 is lower than the validation score on early 2025 (0.823): the 2025 mix, with more clearly weak marketplace leads, is easier to rank, the same effect as in section 36.3.

11.

```

honest_auc = score(cats, nums)[0]

def leak_check(column):
    change = score(cats, nums + [column])[0] - honest_auc
    print(f"{column:<18} AUC change {change:+.3f}")

for column in ["days_in_pipeline", "has_quote", "lead_id"]:
    leak_check(column)

```

```

days_in_pipeline    AUC change +0.052
has_quote            AUC change +0.155
lead_id              AUC change -0.000

```

`lead_id` changes nothing here. Don't take that as a rule: in many systems IDs rise over time or are issued in batches per source, so an ID column can smuggle in time or source information, and it never belongs in a model. A jump of more than about 0.02 from one column, when cross-validation folds vary by a few points, is worth investigating; a jump like `has_quote`'s is almost certainly a leak. But a small change doesn't prove a column is clean: `first_response_hours` only added 0.009. The prediction-moment question is the real test; the score change only tells you where to look first.

12.

```
shared = set(train["email_norm"]) & set(test["email_norm"])
print(len(shared), "emails appear in both train and test")
print(
    "test leads from those emails:",
    test["email_norm"].isin(shared).sum(),
    "of",
    len(test),
)
```

```
17 emails appear in both train and test
test leads from those emails: 17 of 1185
```

Only a small number of test leads share an email with a training lead, because this generator creates mostly one-off enquiries. In real CRMs, repeat enquirers are common, and they matter: if a customer who was won in training enquires again in test, the model can partly recognize the customer instead of learning general patterns. A **group split** (`GroupKFold` or `GroupShuffleSplit` , with the email or company as the group) keeps every lead from one customer on the same side. With a time split, the question becomes whether "has this customer been won before" should be a legitimate feature, computed as of the prediction moment, which it often should.

13.

```
from sklearn.ensemble import HistGradientBoostingClassifier

def make_boosted(categorical, numeric):
    base = make_model(categorical, numeric)
    return Pipeline(
        [
            ("prepare", base.named_steps["prepare"]),
            ("model", HistGradientBoostingClassifier(random_state=36)),
        ]
    )

boosted = make_boosted(cats, nums).fit(train[cats + nums], train["won"])
p = boosted.predict_proba(valid[cats + nums])[:, 1]
print(
    "gradient boosting:  AUC %.3f  log loss %.4f"
    % (roc_auc_score(valid["won"], p), log_loss(valid["won"], p))
)
print("logistic regression: AUC %.3f  log loss %.4f" % score(cats, nums))
```

```
gradient boosting:  AUC 0.785  log loss 0.2075
logistic regression: AUC 0.823  log loss 0.1948
```

With default settings, gradient boosting does not beat logistic regression here. The planted signal in this data is mostly additive (each feature pushes the odds up or down on its own), which is exactly what logistic regression models, and with about 580 wins in training there isn't much data for boosting to find subtle interactions without overfitting. That's a normal, honest result; Chapter 37 tunes boosting properly and shows when it wins. (The pipeline reuses the same preparation steps; `HistGradientBoostingClassifier` accepts the sparse one-hot output.)

14. First, leakage: list every feature and ask whether it exists at the moment of prediction, looking hardest at anything filled in during the ticket's life (priority changes, number of replies, assigned team). Second, the split: were tickets from the same customer or the same incident in both training and validation, and was the split by time? Third, the baseline and the base rate: what does "escalate if the ticket mentions 'refund'" or "escalate if the customer is enterprise" score? An AUC of 0.97 on a messy human process is far above what's normally achievable, so assume a leak until each check passes.

15. The outcomes are no longer independent of the model. High-scoring leads get called first and followed up more, so they're won more often partly *because* they scored high; low-scoring leads get less attention and are lost more often. A model retrained on that data learns the old model's opinions back, strengthened, and can't tell whether a low-scored lead would have been won with a call. This is a **feedback loop**. Keeping a small random sample of leads that are worked in normal order, not by score, gives unbiased data for retraining and for measuring the model's real effect (Chapter 30 on experiments).

16. "The model can't tell which columns were filled in after a lead was won, so 'all the columns' would include some that are the answer in disguise, and it would look excellent in testing and fail on real leads. I'll include every column that's known when we score a lead, and I'll show you the list and why the others were left out."